Protokoll – Zwischenabgabe “TourPlaner”

Lehrveranstaltung: Software Engineering 2 (SE2)
Semesterprojekt: Tour Pl „ anner
Abgabeart: Intermediate (Frontend + Backend-Integration)
Datum: 12.04.2026
Team / Mitwirkende: Abdullah Hakimi, Humza Ajaz
Repository: https://github.com/Abdullah10-swen/tour-planner-team09

---

1. Ziel der Zwischenabgabe

Laut Aufgabenstellung soll für die Intermediate-Abgabe die Kopplung von Angular-Frontend und Spring-Boot-Backend stehen: funktionsfähige Oberfläche für Touren und Tour-Logs mit REST/JSON, ohne dass bereits alle Final-Features (z. B. PostgreSQL, Login, OpenRouteService, Volltextsuche) vollständig umgesetzt sein müssen.

Unser Stand: Dashboard mit Tour-Liste und -details, Formulare für neue/bearbeitete Touren und Logs, Karten-Platzhalter, Anbindung an die REST-API unter /api/tours.

2. Kurzüberblick Technik

Schicht	Technologie	Rolle
Frontend	Angular 21 (Standalone, Signals wo sinnvoll)	UI, Routing, HTTP-Client
API	Spring Boot 4	REST, JSON, CORS
„DAL“ (Intermediate)	In-Memory-Implementierung	Datenhaltung bis Umstellung auf JPA/PostgreSQL im Final

MVVM in Angular: Die View (Templates/Komponenten) ist von der Geschäftslogik über Services (z. B. TourApiService) und Komponenten-State (Signals) getrennt; Formulare nutzen Reactive Forms mit Validierung.

3. UX-Beschreibung (User Experience)

3.1 Zielgruppe und Nutzungskontext

Die Anwendung richtet sich an sportbegeisterte Nutzer, die Touren planen und absolvierte Aktivitäten per Log festhalten. Die Oberfläche soll ruhig, lesbar und mobilfähig sein (responsive Layout, z. B. Sidebar untereinander auf schmalen Viewports).

3.2 Informationsarchitektur / Navigation

- Start: Dashboard (/) – Liste „Meine Touren“, Auswahl einer Tour, Detailbereich mit Kennzahlen und Log-Liste sowie “+ Neue Tour”.
- Neue Tour: /tours/new – Formular links, Karten-Platzhalter rechts; Speichern/Abbrechen in der Kopfzeile.
- Tour bearbeiten: /tours/:id/edit – gleiche Formularstruktur mit vorausgefüllten Daten.
- Neuer Log: /tours/:tourId/logs/new – Kontext der Tour (Name, Kennzahlen), Eingabe der Log-Felder.
- Log bearbeiten: /tours/:tourId/logs/:logId/edit.

Bewusst weggelassen (Intermediate): Login- und Registrierungs-Flows (laut Abgabefokus nicht erforderlich).

3.3 Visuelles Konzept (Kurz)

- Farben: helles Hintergrund-Beige/Creme, Waldgrün für Primäraktionen, dezente Grautöne für Flächen und Rahmen.
- Typo: Serif für App-Namen/Überschriften, Sans-serif für UI-Texte (Orientierung an Mockup).
- Komponenten: stark abgerundete Buttons und Eingabefelder; wiederverwendbare Elemente z. B. Header (Suche, „Neue Tour“) und Kennzahlen-Karten (MetricCard).

3.4 Feedback & Validierung

Pflichtfelder in Formularen mit Reactive Forms; bei ungültiger Eingabe kein Absturz, sondern Hinweise bzw. deaktiviertes Speichern wo sinnvoll. Fehlertext, wenn das Backend nicht erreichbar ist.

4. Wireframes (Pflicht laut Checkliste)

Die folgenden Abbildungen 1–5 im Abschnitt 4.1 zeigen die geplanten Oberflächen als High-Fidelity-Mockups – das entspricht den geforderten Wireframes bzw. dem UX-Design vor der Implementierung. Figma-Link

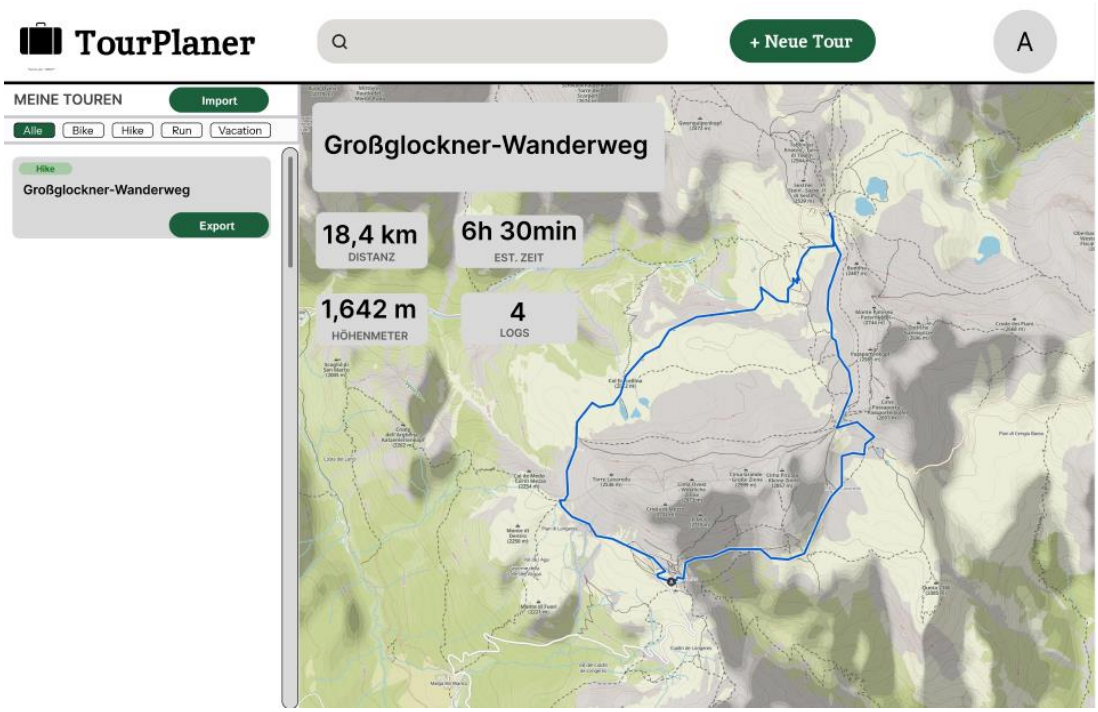
Desweiteren ergänzen wir einen illustrativen Vergleich zwischen Design (Mockup) und der laufenden Angular-Oberfläche.

Priorität im Projekt: Ziel der Zwischenabgabe war die funktionale Umsetzung und die Erfüllung der Anforderungen laut Spezifikation/Checkliste – nicht die pixelgenaue Nachbildung des Mockups. Dieses Ziel wurde erreicht. Abweichungen zum Figma-Layout sind daher beabsichtigt möglich und keine Qualitätsmängel per se. Für die Finalabgabe gilt dieselbe Linie: Spezifikation und fachliche Vollständigkeit stehen über 1:1-Optik zum Mockup; das Mockup bleibt die Orientierung für UX und Branding.

Abbildung 1 – Dashboard „Meine Touren“

Zweck: Übersicht aller Touren, Filter nach Transporttyp, Suche, CTA „Neue Tour“, ausgewählte Tour mit Detail-Kennzahlen und Log-Liste; Karten-Platzhalter.

Mockup:



Implementierung:

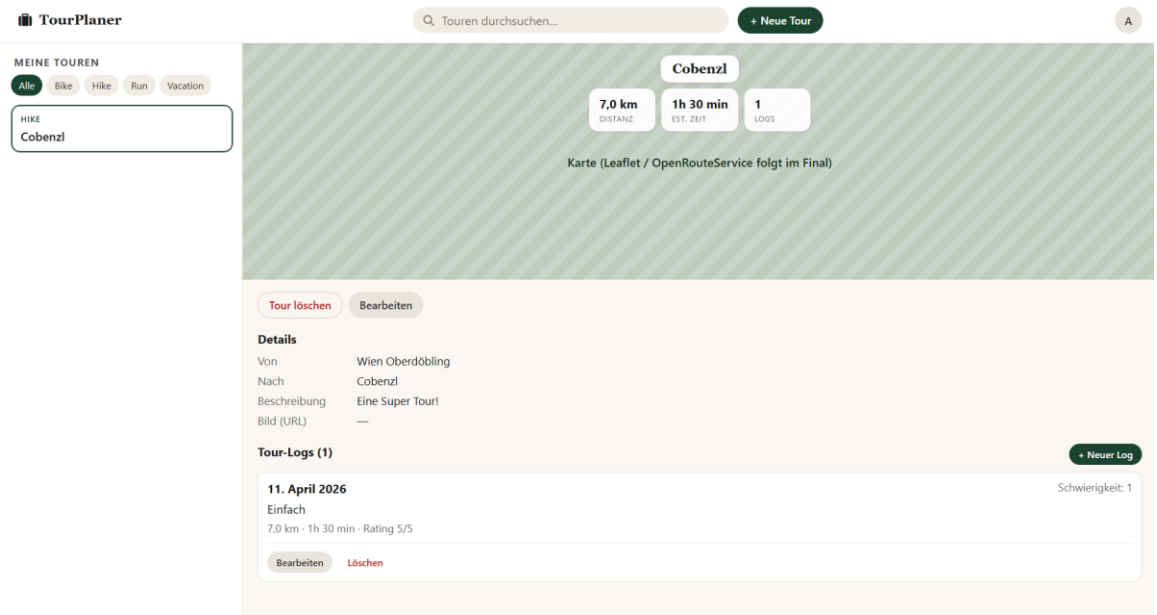
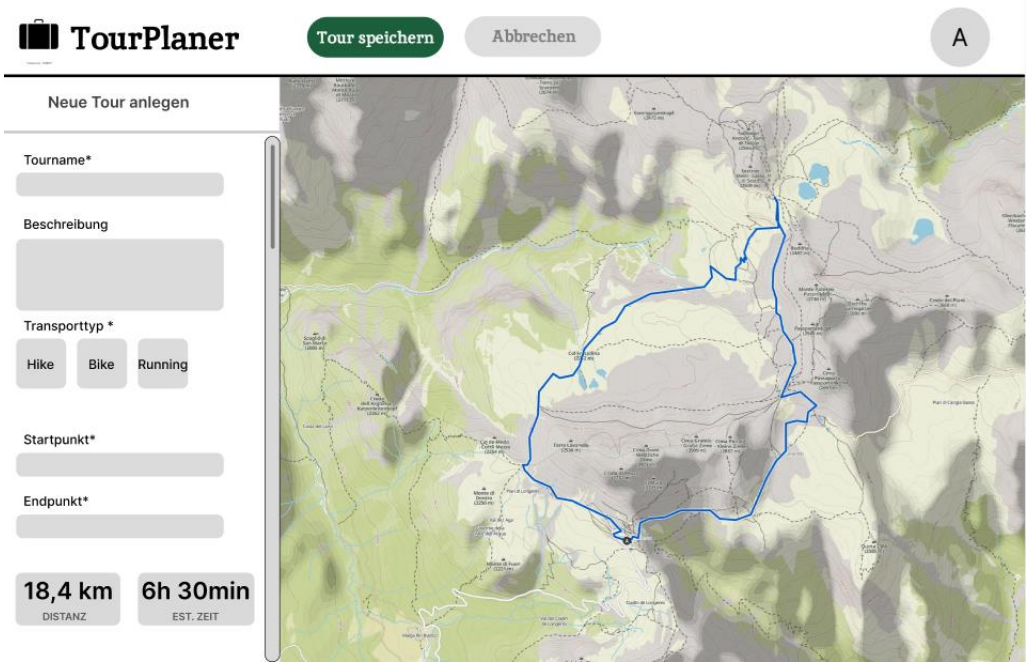


Abbildung 2 – Tour anlegen / bearbeiten

Zweck: Erfassung Name, Beschreibung, Transport, Start/Ziel, Distanz/Zeit, Bild-URL, Routen-Info; Speichern/Abbrechen; Karten-Platzhalter.

Mockup:

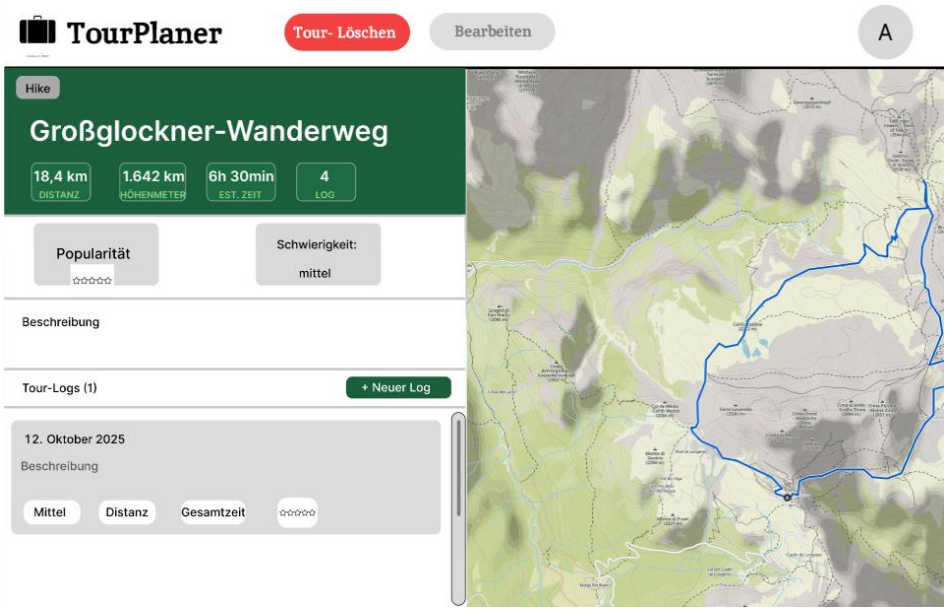


Implementierung:

Abbildung 3 – Tour-Detail mit Logs

Zweck: Fokus auf Beschreibung, Metriken, Log-Liste mit Aktionen.

Mockup:

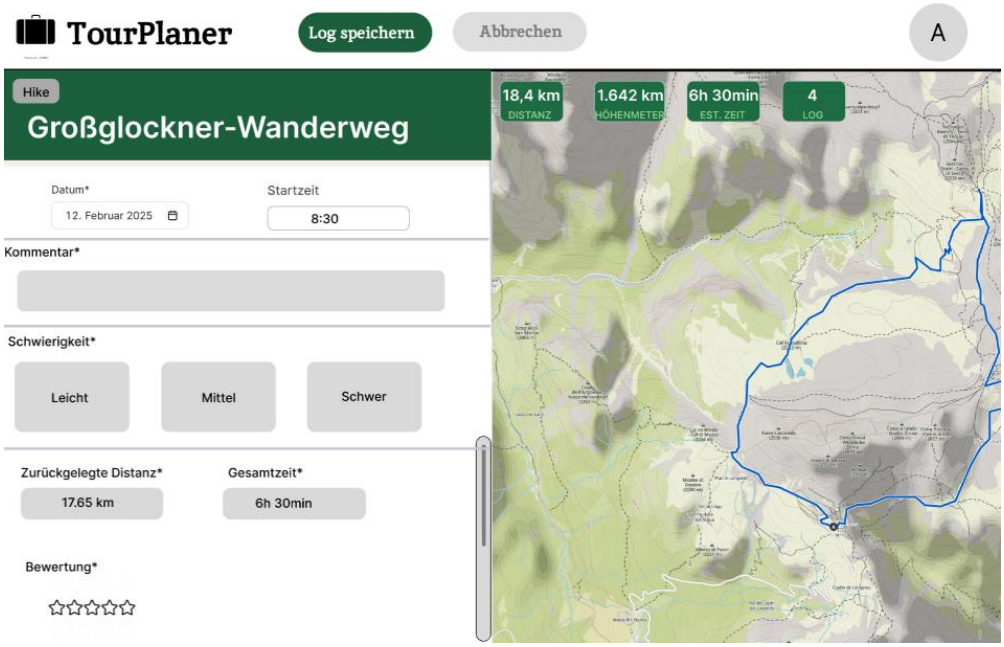


Implementierung: Siehe Abbildung 1

Abbildung 4 – Log anlegen / bearbeiten

Zweck: Datum/Zeit, Kommentar, Schwierigkeit, Distanz, Zeit, Bewertung; Kontext der Tour sichtbar.

Mockup:



Implementierung:

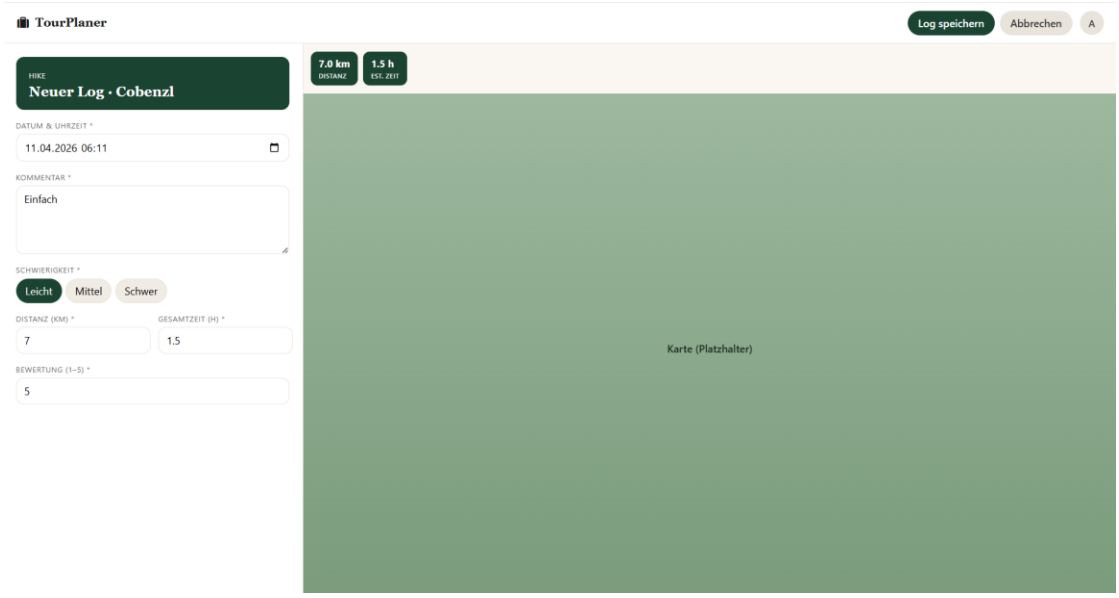


Abbildung 5/6 – Login / Registrierung (nur Dokumentation, nicht Teil von Intermediate)

Hinweis: Für die Intermediate-Abgabe nicht implementiert; Mockup dient der Vollständigkeit des UI-Konzepts und dem Final.

5. Backend-Integration (Kurz)

- Basis-URL: konfigurierbar im Frontend (api-base.ts), Standard http://localhost:8080/api.
- Ressourcen: GET/POST/PUT/DELETE für Touren; verschachtelte Pfade für Logs (…/tours/{id}/logs/…).
- Datenformat: JSON; DTOs im Backend entkoppeln die REST-Schicht vom Domain-Modell.

6. Entscheidungen & Abgrenzungen

Thema	Entscheidung
Persistenz Intermediate	In-Memory statt DB, um Fokus auf UI + API zu legen
Karte	Platzhalter statt Leaflet/OpenRouteService (Final-Pflicht)
Auth	Kein Login in dieser Abgabestufe
Dateiname Service	Korrekte Endung .java für plattformübergreifendes Build (Git/Linux)

7. Probleme / Lessons Learned

Beim lokalen Test mussten Backend (`localhost:8080`) und Frontend (`localhost:4200`) gleichzeitig laufen; CORS wurde im Spring-Controller mit `@CrossOrigin` gelöst, damit der Browser die API aufrufen darf.

Unter Windows ist die Dateiendung `.Java` vs. `.java` für den Compiler irrelevant, GitHub (Linux) unterscheidet sie aber – der Service musste im Git-Index einmal per `git mv` korrekt auf `.java` umbenannt werden, sonst wirkte der Pfad auf dem Remote „falsch“.

Im Angular-Projekt (ohne klassisches Zone.js) aktualisierte sich die Oberfläche nach manchen HTTP-Responses nicht sofort; mit Signals für asynchron geladene Daten (z. B. Tour im Log-Formular) ließ sich das zuverlässig beheben.

8. Zeitaufwand (Schätzung / Ist)

Arbeitspaket	Stunden (Ist/Schätzung)
Backend REST + DTO	10/12
Angular Dashboard + Routing	18/22
Formulare + Validierung	12/15
Integration, CORS, Fehlersuche, UI-Feinheiten	6/8
Wireframes / Protokoll / README	8/9
Summe	54/66

9. Git-Historie

Die Entwicklungsgeschichte ist über das Git-Repository nachvollziehbar (Commits, Branches, Merges). Eine Kopie der Historie ins PDF ist laut Aufgabenstellung nicht nötig; Verweis auf das Repo genügt.